

beim rumspielen habe ich entdeckt dass er manchmal trotzdem wieder 89.92 aber ohne euro anzeigt.. (es gibt bei den neuen subscription die möglichkeit optionen auszuwählen)

Ah! Das ist ein **hervorragender** Fund beim Testen! Das ist ein echtes Senior-Developer-Problem, das du da gerade entdeckt hast.

Lass mich dir genau erklären, was da im Hintergrund passiert, denn das ist eine spannende Geschichte für dein Interview:

Wenn du eine neue Option auswählst, ändert sich der Preis (z. B. auf 99.92). React ist extrem effizient (das sogenannte *Virtual DOM*). Es lädt nicht die ganze Seite neu und es berührt auch nicht das Euro-Zeichen, weil sich das ja nicht geändert hat. React updatet **ausschließlich** den Text-Node mit der Zahl.

Unser Problem: Wir haben das Euro-Zeichen beim ersten Durchlauf in unserem Skript gelöscht (`node.nodeValue = text.replace("€", "");`), um es hinter die Zahl zu schieben. Wenn React nun die Zahl auf 99.92 ändert, springt unser Skript an und schaut sich die Node an. Es sieht 99.92. Es sucht nach einem € in der Nähe, findet aber keines mehr (weil wir es ja vorher gelöscht haben!). Deshalb denkt das Skript: *"Oh, das ist nur irgendeine Zahl, kein Preis. Ich mache nichts."*

Die Lösung: Ein unsichtbarer Marker

Wir müssen dem HTML-Element (dem `<span>`, in dem der Preis steht) beim ersten Mal einen "Stempel" aufdrücken: *"Hey, hier wohnt ein Preis!"* Wenn React dann später nur die Zahl ändert, sieht unser Skript den Stempel am übergeordneten Element und weiß sofort: *"Ah, das ist der Preis, ich muss wieder ein Komma und ein €-Zeichen dranhängen!"*

Wir passen den oberen Teil unserer `processTextNode`-Funktion an. Tausche den Block --- **A. WÄHRUNGSFORMAT** --- durch diesen hier aus:

JavaScript

```
// --- A. WÄHRUNGSFORMAT (React Split-Node & Re-Render Fix) ---
// Fall 1: Wir sehen das Euro-Zeichen (Initiales Laden oder kompletter Re-Render)
if (text.trim() === "€") {
  let nextNode = node.nextSibling;
  while (nextNode && nextNode.nodeType === 3 && nextNode.nodeValue.trim() === "") {
    nextNode = nextNode.nextSibling;
  }
  if (nextNode && nextNode.nodeType === 3) {
    const nextText = nextNode.nodeValue;
    const numberRegex = /^\\s*(\\d+)\\.\\d{2}\\s*$/;
    if (numberRegex.test(nextText)) {
```

```

// WICHTIG: Wir markieren das Eltern-Element als "Preis-Container"
if (node.parentNode) {
  node.parentNode.setAttribute('data-is-price', 'true');
}
node.nodeValue = text.replace("€", "");
nextNode.nodeValue = nextText.replace(numberRegex, "$1,$2 €");
return;
}
}
}

```

```

// Fall 2: React hat NUR die Zahl geupdatet (z.B. bei Optionswechsel)
// Wir prüfen, ob es eine isolierte Zahl ist UND ob das Elternteil unseren Stempel hat
const standaloneNumberRegex = /^(\s*(\d+)\.(\d{2}))\s*$/;
if (standaloneNumberRegex.test(text)) {
  if (node.parentNode && node.parentNode.getAttribute('data-is-price') === 'true') {
    text = text.replace(standaloneNumberRegex, "$1,$2 €");
  }
}
}

```

```

// Fall 3: Fallback für zusammenhängende Nodes (z.B. "€89.92" in einem String)
text = text.replace(/(€\s*(\d+)\.(\d{2}))/g, "$1,$2 €");

```

Warum das so elegant ist:

Mit `node.parentNode.setAttribute('data-is-price', 'true')` injizieren wir ein unsichtbares HTML-Attribut in das Element. Egal, wie oft React nun im Inneren die Zahl austauscht, das übergeordnete `span`-Element behält dieses Attribut. Unser Skript (Fall 2) greift nun dieses Attribut ab und repariert die Zahl zuverlässig bei jedem Klick!

Tausch diesen Block einfach in deinem Code aus. Wenn du jetzt mit den Optionen herumspielst, sollte das € stabil stehen bleiben!